

Splitwise - How it works

Home notes + DESIGN.md + source code | compact print

In one line - Friends share expenses -> system tracks who owes whom -> simplify into fewer payments.

Story - how this LLD works

Goa trip: Shubh pays 100 for lunch with Bob (equal) -> Bob owes Shubh 50. Next day Bob pays 100 for lunch with Tom -> Tom owes Bob 50. Instead of Bob collecting from Tom and paying Shubh, simplify: Tom pays Shubh 50 directly. Bob is settled as the middle person.

Who does what

Group: A trip or circle. Holds members, expenses, and each person's BalanceSheet.

User: A friend (frozen id+name - can be dict key).

Expense: One bill: who paid, how much, how it was split.

Split: One person's share of that bill.

BalanceSheet: For one user: total paid, total spent, and map of 'I owe X / X owes me'.

SplitStrategy: Equal or Percentage math.

GroupService: Simple API you call from outside (facade).

DebtSimplificationService: Uses heaps to reduce pairwise debts to net settlements.

Main pieces

Group

- members
- expenses
- balanceSheets

Expense + Splits

- paidBy
- shares

BalanceSheet

- owes map
- total paid/spent

Simplifier

- nets + heaps
- fewer payments

Step-by-step at runtime

1. createGroup('Goa Trip', [Shubh, Bob, Tom]) - each gets empty BalanceSheet.
2. addExpense Lunch1: Shubh paid 100, split equal [Shubh, Bob].
3. Balance update: Bob owes Shubh 50 (and reverse +50 on Shubh's sheet).
4. addExpense Lunch2: Bob paid 100, split equal [Bob, Tom] -> Tom owes Bob 50.
5. simplifyDebts: compute net per person, clear pairwise, match biggest creditor with biggest debtor.
6. Result: Tom -> Shubh 50. Bob net zero.
7. printBalances shows the friendly summary.

Why this design (plain words)

Why Strategy + Factory for splits? - Equal today, percentage tomorrow - same addExpense path.

Why Facade GroupService? - Caller shouldn't wire ExpenseService + BalanceSheet + Simplifier manually.

Why simplify? - Fewer UPI transfers between friends. Totals stay fair; paths get shorter.

Remember

- Invariant: if A shows +50 for B, B shows -50 for A.
- Payer who is also participant still gets expense share but no self-debt row.
- Floating point (100/3) is a real interview gotcha - use cents in production.
- Participant not in group -> KeyError (validation gap).

OOP in this project

Encapsulation - You update balances through service methods, not raw dicts from outside.

Abstraction - SplitStrategy hides equal vs percentage math.

Inheritance - EqualSplit / PercentageSplit implement the same interface.

Polymorphism - factory returns a strategy; expense code just calls split().

DESIGN.md - full file

Complete markdown from lld/splitwise/DESIGN.md (Q&A for learning).

Full DESIGN.md (same file as in this folder) - DESIGN.md

Designing Splitwise - LLD Q&A

Run: `python -m lld.splitwise.main`

Patterns: Strategy · Factory · Repository · Facade · Debt simplification

Q1. What problem are we solving?

A. Design a group expense-sharing system that:

Creates groups with members

Records expenses (one person pays, cost split among participants)

Supports multiple split types (equal, percentage)

Maintains balance sheets (who owes whom)

Simplifies debts into a minimal set of net settlements

Q2. What are the core entities?

- Entity - Responsibility
- User - Frozen identity (id, name) - hashable dict key
- Group - Members, expenses, per-user balance sheets
- Expense - Description, amount, paid_by, splits, split_type
- Split - One participant's share (user, amount)
- BalanceSheet - total_paid, total_expense, pairwise balances
- Balance semantics: Positive = other owes you; Negative = you owe them.
- Invariant: $A.balances[B] == -B.balances[A]$.

Q3. How is the package structured?

```
splitwise/
|-- model/      # User, Group, Expense, Split, BalanceSheet
|-- strategy/   # EqualSplit, PercentageSplit
|-- factory/    # SplitStrategyFactory
|-- repository/ # GroupRepository (ABC) + InMemory
|-- service/    # GroupService, ExpenseService, BalanceSheetService,
                # DebtSimplificationService
```

GroupService is the public facade; other services hide complexity.

Q4. Why Strategy + Factory for splits?

A. Split math varies; ExpenseService should not care how shares are computed.

```
strategy = SplitStrategyFactory.get_strategy(split_type)
splits = strategy.split(amount, participants, metadata)
```

- Type - Behavior
- EQUAL - $\text{amount} / \text{len}(\text{participants})$
- PERCENTAGE - Share by %; must sum to 100
- New split type (EXACT, SHARES) = new strategy + factory branch - no change to ExpenseService.

Q5. What is the add-expense flow?

```
GroupService.add_expense(...)
-> ExpenseService
  -> factory -> strategy.split(...)
  -> create Expense, append to group
  -> BalanceSheetService.update_balances(...)
    -> paid_by: +total_paid
    -> each participant: +total_expense
    -> pairwise: debtor owes payer (skip if same person)
```

Demo:

Lunch Day-1: Shubh pays 100 for [Shubh, Bob] -> Bob owes Shubh 50

Lunch Day-2: Bob pays 100 for [Bob, Tom] -> Tom owes Bob 50

Q6. How does debt simplification work?

A. Greedy net settlement with heaps:

Compute each user's net (sum of pairwise balances)

Clear pairwise balances

Creditors (net > 0) and debtors (net < 0) go into heaps

Match largest creditor with largest debtor; settle $\min(\text{credit}, |\text{debit}|)$

Re-push remainders until done

After simplify on demo: Tom pays Shubh 50 directly; Bob (intermediary) is settled.

Does not change totals owed - only reduces number of transactions (at most $n-1$ for n users).

Q7. Why Repository?

A. GroupRepository ABC + InMemoryGroupRepository decouples storage. Swap for DB later without changing services.

Q8. How do you extend this design?

- Add... - How
- EXACT / SHARES split - New SplitStrategy + enum + factory
- Settlements - New service to mark debts paid
- Multi-currency - Currency on expense + conversion before balances
- Edit/delete expense - Reverse balance entries, then re-apply

Q9. Common interview follow-ups

Q. Why is User frozen?

Hashable - safe as dict key in balance sheets.

Q. Floating point on equal split of 100 among 3?

33.33x3 != 100. Production uses cents or remainder distribution.

Q. What if participant is not in the group?

get_balance_sheet raises KeyError - validation gap.

Q. Greedy vs optimal min-cash-flow?

Full optimal settlement is harder (NP-hard in general graphs). Greedy on nets is practical and interview-friendly.

Source code - Ild/splitwise/

17 Python files below (compact font to save pages). Read with the notes: find the class name from the cast, then open that file here.

- Tip: start at main.py, then service/, then model/ + strategy/.

FILE: main.py (51 lines)

```

1| from .enums.split_type import SplitType
2| from .model.user import User
3| from .repository.in_memory_group_repository import InMemoryGroupRepository
4| from .service.balance_sheet_service import BalanceSheetService
5| from .service.debt_simplification_service import DebtSimplificationService
6| from .service.expense_service import ExpenseService
7| from .service.group_service import GroupService
8|
9|
10| def main() -> None:
11|     shubh = User("u1", "Shubh")
12|     bob = User("u2", "Bob")
13|     tom = User("u3", "Tom")
14|     jake = User("u4", "Jake")
15|
16|     repo = InMemoryGroupRepository()
17|     balance_sheet_service = BalanceSheetService()
18|     expense_service = ExpenseService(balance_sheet_service)
19|     simplification_service = DebtSimplificationService()
20|
21|     group_service = GroupService(repo, expense_service, simplification_service)
22|
23|     goa_group_id = group_service.create_group("Goa Trip", [shubh, bob, tom])
24|     group_service.create_group("Non-Group Expenses", [shubh, bob, tom, jake])
25|
26|     group_service.add_expense(
27|         goa_group_id,
28|         "Lunch Day-1",
29|         100,
30|         shubh,
31|         [shubh, bob],
32|         SplitType.EQUAL,
33|         None,
34|     )
35|
36|     group_service.add_expense(
37|         goa_group_id,
38|         "Lunch Day-2",
39|         100,
40|         bob,
41|         [bob, tom],
42|         SplitType.EQUAL,
43|         None,
44|     )
45|
46|     group_service.simplify_debts(goa_group_id)
47|     group_service.print_balances(goa_group_id)
48|
49|
50| if __name__ == "__main__":
51|     main()

```

FILE: service/balance_sheet_service.py (19 lines)

```

1| from ..model.group import Group
2| from ..model.split import Split
3| from ..model.user import User
4|
5|
6| class BalanceSheetService:
7|     def update_balances(
8|         self, group: Group, paid_by: User, splits: list[Split]
9|     ) -> None:
10|         total_amount = sum(split.amount for split in splits)
11|         group.get_balance_sheet(paid_by).add_total_paid(total_amount)
12|
13|         for split in splits:
14|             user = split.user
15|             amt = split.amount
16|             group.get_balance_sheet(user).add_total_expense(amt)
17|             if user is not paid_by:
18|                 group.get_balance_sheet(user).add_balance(paid_by, -amt)
19|                 group.get_balance_sheet(paid_by).add_balance(user, amt)

```

FILE: service/debt_simplification_service.py (74 lines)

```

1| import heapq
2| from typing import Dict, List
3|
4| from ..model.group import Group
5| from ..model.user import User
6|
7|
8| class _CreditorEntry:
9|     def __init__(self, user: User, net: float) -> None:
10|         self.user = user
11|         self.net = net
12|
13|     def __lt__(self, other: "_CreditorEntry") -> bool:
14|         return self.net > other.net
15|
16|
17| class _DebtorEntry:
18|     def __init__(self, user: User, net: float) -> None:

```

```

19|     self.user = user
20|     self.net = net
21|
22| def __lt__(self, other: "_DebtorEntry") -> bool:
23|     return self.net < other.net
24|
25|
26| class DebtSimplificationService:
27|     def simplify_debts(self, group: Group) -> None:
28|         users: List[User] = list(group.members)
29|         sheets = group.balance_sheets
30|
31|         net_balances: Dict[User, float] = {}
32|         for user in users:
33|             net = 0.0
34|             balances = sheets[user].balances
35|             for amount in balances.values():
36|                 net += amount
37|             net_balances[user] = net
38|             sheets[user].clear_balances()
39|
40|         creditors: List[_CreditorEntry] = []
41|         debtors: List[_DebtorEntry] = []
42|
43|         for user in users:
44|             net = net_balances[user]
45|             if net > 0:
46|                 heapq.heappush(creditors, _CreditorEntry(user, net))
47|             elif net < 0:
48|                 heapq.heappush(debtors, _DebtorEntry(user, net))
49|
50|         while creditors and debtors:
51|             creditor_entry = heapq.heappop(creditors)
52|             debtor_entry = heapq.heappop(debtors)
53|             creditor = creditor_entry.user
54|             debtor = debtor_entry.user
55|
56|             credit_amount = net_balances[creditor]
57|             debit_amount = net_balances[debtor]
58|
59|             settled_amount = min(credit_amount, -debit_amount)
60|
61|             sheets[creditor].add_balance(debtor, settled_amount)
62|             sheets[debtor].add_balance(creditor, -settled_amount)
63|
64|             net_balances[creditor] = credit_amount - settled_amount
65|             net_balances[debtor] = debit_amount + settled_amount
66|
67|             if net_balances[creditor] > 0:
68|                 heapq.heappush(
69|                     creditors, _CreditorEntry(creditor, net_balances[creditor])
70|                 )
71|             if net_balances[debtor] < 0:
72|                 heapq.heappush(
73|                     debtors, _DebtorEntry(debtor, net_balances[debtor])
74|                 )

```

FILE: service/expense_service.py (32 lines)

```

1| from typing import Dict, List, Optional
2|
3| from ..enums.split_type import SplitType
4| from ..factory.split_strategy_factory import SplitStrategyFactory
5| from ..model.expense import Expense
6| from ..model.group import Group
7| from ..model.user import User
8| from .balance_sheet_service import BalanceSheetService
9|
10|
11| class ExpenseService:
12|     def __init__(self, balance_sheet_service: BalanceSheetService) -> None:
13|         self._balance_sheet_service = balance_sheet_service
14|
15|     def add_expense(
16|         self,
17|         group: Group,
18|         description: str,
19|         amount: float,
20|         paid_by: User,
21|         participants: List[User],
22|         split_type: SplitType,
23|         metadata: Optional[Dict[User, float]],
24|     ) -> None:
25|         if metadata is None:
26|             metadata = {}
27|         strategy = SplitStrategyFactory.get_strategy(split_type)
28|         splits = strategy.split(amount, participants, metadata)
29|         expense = Expense(description, amount, paid_by, splits, split_type)
30|         group.add_expense(expense)
31|
32|         self._balance_sheet_service.update_balances(group, paid_by, splits)

```

FILE: service/group_service.py (87 lines)

(cont.) service/group_service.py

```

1| import uuid
2| from typing import Dict, List, Optional
3|
4| from ..enums.split_type import SplitType
5| from ..model.group import Group
6| from ..model.user import User
7| from ..repository.group_repository import GroupRepository
8| from ..debt_simplification_service import DebtSimplificationService
9| from ..expense_service import ExpenseService
10|
11|
12| class GroupService:
13|     def __init__(
14|         self,
15|         repo: GroupRepository,
16|         expense_service: ExpenseService,
17|         simplifier: DebtSimplificationService,
18|     ) -> None:
19|         self._repo = repo
20|         self._expense_service = expense_service
21|         self._simplifier = simplifier
22|
23|     def create_group(self, name: str, members: List[User]) -> str:
24|         group_id = str(uuid.uuid4())
25|         group = Group(group_id, name)
26|         for member in members:
27|             group.add_member(member)
28|
29|         self._repo.save(group)
30|         return group_id
31|
32|     def add_member(self, group_id: str, user: User) -> None:
33|         self._get(group_id).add_member(user)
34|
35|     def add_expense(
36|         self,
37|         group_id: str,
38|         description: str,
39|         amount: float,
40|         paid_by: User,
41|         participants: List[User],
42|         split_type: SplitType,
43|         meta: Optional[Dict[User, float]],
44|     ) -> None:
45|         self._expense_service.add_expense(
46|             self._get(group_id),
47|             description,
48|             amount,
49|             paid_by,
50|             participants,
51|             split_type,
52|             meta,
53|         )
54|
55|     def simplify_debts(self, group_id: str) -> None:
56|         self._simplifier.simplify_debts(self._get(group_id))
57|
58|     def print_balances(self, group_id: str) -> None:
59|         group = self._get(group_id)
60|         for user in group.members:
61|             sheet = group.get_balance_sheet(user)
62|
63|             owe = 0.0
64|             get = 0.0
65|             for value in sheet.balances.values():
66|                 if value < 0:
67|                     owe += -value
68|                 else:
69|                     get += value
70|
71|             print(
72|                 f"                ? {user.name}\n"
73|                 f"                Paid: {sheet.total_paid:.2f} "
74|                 f"Expense: {sheet.total_expense:.2f}\n"
75|                 f"                You owe: {owe:.2f}, You get: {get:.2f}"
76|             )
77|
78|             for other, val in sheet.balances.items():
79|                 direction = "<- get" if val > 0 else "-> owe"
80|                 print(f" {direction} {abs(val):.2f} {other.name}")
81|             print("-----")
82|
83|     def _get(self, group_id: str) -> Group:
84|         group = self._repo.find_by_id(group_id)
85|         if group is None:
86|             raise ValueError(f"Group not found: {group_id}")
87|         return group

```

FILE: model/balance_sheet.py (47 lines)

(cont.) model/balance_sheet.py

```

1| from typing import Dict
2|
3| from .user import User
4|
5|
6| class BalanceSheet:
7|     def __init__(self) -> None:
8|         self.total_paid: float = 0.0
9|         self.total_expense: float = 0.0
10|         self.balances: Dict[User, float] = {}
11|
12|     def add_total_paid(self, amount: float) -> None:
13|         self.total_paid += amount
14|
15|     def add_total_expense(self, amount: float) -> None:
16|         self.total_expense += amount
17|
18|     def add_balance(self, other: User, amount: float) -> None:
19|         self.balances[other] = self.balances.get(other, 0.0) + amount
20|         if abs(self.balances[other]) < 1e-6:
21|             del self.balances[other]
22|
23|     def clear_balances(self) -> None:
24|         self.balances.clear()
25|
26|     def print(self, me: User) -> None:
27|         you_owe = 0.0
28|         you_get_back = 0.0
29|         for amount in self.balances.values():
30|             if amount < 0:
31|                 you_owe += -amount
32|             else:
33|                 you_get_back += amount
34|
35|         print(f"? Balance sheet of : {me.name}")
36|         print(f"Total You Paid : {self.total_paid}")
37|         print(f"Total Expense : {self.total_expense}")
38|         print(f"Total You Owe : {you_owe}")
39|         print(f"Total You Get Back : {you_get_back}")
40|
41|         for other, amount in self.balances.items():
42|             if amount > 0:
43|                 print(f"You get back {amount} from {other.name}")
44|             elif amount < 0:
45|                 print(f"You owe {-amount} to {other.name}")
46|         print("-----")
47|         print("-----")

```

FILE: model/expense.py (22 lines)

```

1| from dataclasses import dataclass
2| from typing import List
3|
4| from ..enums.split_type import SplitType
5| from .split import Split
6| from .user import User
7|
8|
9| @dataclass
10| class Expense:
11|     description: str
12|     amount: float
13|     paid_by: User
14|     splits: List[Split]
15|     split_type: SplitType
16|
17|     def __str__(self) -> str:
18|         return (
19|             f"Expense(description={self.description!r}, amount={self.amount}, "
20|             f"paid_by={self.paid_by}, splits={self.splits}, "
21|             f"split_type={self.split_type})"
22|         )

```

FILE: model/group.py (24 lines)

```

1| from typing import Dict, List
2|
3| from .balance_sheet import BalanceSheet
4| from .expense import Expense
5| from .user import User
6|
7|
8| class Group:
9|     def __init__(self, id: str, name: str) -> None:
10|         self.id = id
11|         self.name = name
12|         self.members: List[User] = []
13|         self.expenses: List[Expense] = []
14|         self.balance_sheets: Dict[User, BalanceSheet] = {}
15|
16|     def add_member(self, user: User) -> None:
17|         self.members.append(user)
18|         self.balance_sheets.setdefault(user, BalanceSheet())
19|
20|     def add_expense(self, expense: Expense) -> None:
21|         self.expenses.append(expense)
22|
23|     def get_balance_sheet(self, user: User) -> BalanceSheet:
24|         return self.balance_sheets[user]

```

FILE: model/split.py (12 lines)

```

1| from dataclasses import dataclass
2|
3| from .user import User
4|
5|
6| @dataclass
7| class Split:
8|     user: User
9|     amount: float
10|
11|     def __str__(self) -> str:
12|         return f"Split(user={self.user}, amount={self.amount})"

```

FILE: model/user.py (10 lines)

```

1| from dataclasses import dataclass
2|
3|
4| @dataclass(eq=False, frozen=True)
5| class User:
6|     id: str
7|     name: str
8|
9|     def __str__(self) -> str:
10|         return f"User(id={self.id!r}, name={self.name!r})"

```

FILE: strategy/equal_split_strategy.py (19 lines)

```

1| from typing import Dict, List
2|
3| from ..model.split import Split
4| from ..model.user import User
5| from .split_strategy import SplitStrategy
6|
7|
8| class EqualSplitStrategy(SplitStrategy):
9|     def split(
10|         self,
11|         total_amount: float,
12|         participants: List[User],
13|         metadata: Dict[User, float],
14|     ) -> List[Split]:
15|         share = total_amount / len(participants)
16|         splits: List[Split] = []
17|         for user in participants:
18|             splits.append(Split(user, share))
19|         return splits

```

FILE: strategy/percentage_split_strategy.py (24 lines)

```

1| from typing import Dict, List
2|
3| from ..model.split import Split
4| from ..model.user import User
5| from .split_strategy import SplitStrategy
6|
7|
8| class PercentageSplitStrategy(SplitStrategy):
9|     def split(
10|         self,
11|         total_amount: float,
12|         participants: List[User],
13|         metadata: Dict[User, float],
14|     ) -> List[Split]:
15|         total_percent = sum(metadata.values())
16|         if total_percent != 100.0:
17|             raise ValueError("Total percent should be 100")
18|
19|         splits: List[Split] = []
20|         for user in participants:
21|             splits.append(
22|                 Split(user, total_amount * metadata.get(user, 0.0) / 100)
23|             )
24|         return splits

```

FILE: strategy/split_strategy.py (16 lines)

```

1| from abc import ABC, abstractmethod
2| from typing import Dict, List
3|
4| from ..model.split import Split
5| from ..model.user import User
6|
7|
8| class SplitStrategy(ABC):
9|     @abstractmethod
10|     def split(
11|         self,
12|         total_amount: float,
13|         participants: List[User],
14|         metadata: Dict[User, float],
15|     ) -> List[Split]:
16|         pass

```

FILE: factory/split_strategy_factory.py (14 lines)

(cont.) factory/split_strategy_factory.py

```
1| from ..enums.split_type import SplitType
2| from ..strategy.equal_split_strategy import EqualSplitStrategy
3| from ..strategy.percentage_split_strategy import PercentageSplitStrategy
4| from ..strategy.split_strategy import SplitStrategy
5|
6|
7| class SplitStrategyFactory:
8|     @staticmethod
9|     def get_strategy(split_type: SplitType) -> SplitStrategy:
10|         if split_type == SplitType.EQUAL:
11|             return EqualSplitStrategy()
12|         if split_type == SplitType.PERCENTAGE:
13|             return PercentageSplitStrategy()
14|         raise ValueError(f"Unsupported split type: {split_type}")
```

FILE: repository/group_repository.py (14 lines)

```
1| from abc import ABC, abstractmethod
2| from typing import Optional
3|
4| from ..model.group import Group
5|
6|
7| class GroupRepository(ABC):
8|     @abstractmethod
9|     def find_by_id(self, id: str) -> Optional[Group]:
10|         pass
11|
12|     @abstractmethod
13|     def save(self, group: Group) -> None:
14|         pass
```

FILE: repository/in_memory_group_repository.py (15 lines)

```
1| from typing import Dict, Optional
2|
3| from ..model.group import Group
4| from .group_repository import GroupRepository
5|
6|
7| class InMemoryGroupRepository(GroupRepository):
8|     def __init__(self) -> None:
9|         self._store: Dict[str, Group] = {}
10|
11|     def find_by_id(self, id: str) -> Optional[Group]:
12|         return self._store.get(id)
13|
14|     def save(self, group: Group) -> None:
15|         self._store[group.id] = group
```

FILE: enums/split_type.py (6 lines)

```
1| from enum import Enum
2|
3|
4| class SplitType(Enum):
5|     EQUAL = "EQUAL"
6|     PERCENTAGE = "PERCENTAGE"
```